# FraudGuard - Super Easy Viva Guide

Everything Explained in Everyday Simple English for Presentation & Viva

Minor Project-I (AD-508) | SISTec Dept. of AI & Data Science | Team: Wasil, Tayyab, Haseeb, Sufiyan

> 🎯 **Why This Guide Exists**
>
> We wrote this guide without any confusing bookish words or complicated math terms. Every concept is broken down into simple, real-life examples so that **any team member can speak confidently and answer any question without getting stuck**.

## Part 1: What is FraudGuard in 2 Simple Lines?

If the teacher asks: "*Tell me in simple words, what does your project do?*"

> 🗣️ **What You Say:**
>
> **"Ma'am, FraudGuard is a real-time security system for UPI and bank payments. When someone makes a payment, our system checks it in less than 2 milliseconds using Machine Learning. If it looks like a fraud attack, it immediately flags it and explains in simple words why it was blocked."**

### Why do banks need this? Why do old bank rules fail?

- **Old Way (Simple Rules):** Old banks set fixed rules like: "*If a transfer is more than ₹50,000, stop it.*"
- **How Thieves Beat It:** A smart hacker simply sends ₹49,000 twice! The old rule never triggers.
- **The FraudGuard Way:** Our system does not just look at the amount. It looks at the **behavior pattern**: "*Did this person have ₹50,000 and suddenly their entire balance dropped to zero in 2 seconds?*" Thieves cannot hide this pattern.

## Part 2: The 5 Simple Parts of the Project

If the teacher asks: "*Walk me through the system architecture.*"

| Component | Technology | Simple Meaning (What it actually does) |
|---|---|---|
| **1. Frontend (Website)** | HTML, CSS, Bootstrap, Chart.js | The clean website where bank officers can test payments on a simulator, see risk gauges, and review fraud alerts. |
| **2. Backend Server** | Python & Django | The main brain. It receives the payment details from the website, checks them, calls the AI model, and sends back the answer. |
| **3. Smart Math Clues** | Feature Engineering | Two quick formulas that check if the sender and receiver account balances make sense. |
| **4. The ML Model** | XGBoost 2.0.3 | The trained AI model that gives a Risk Score from 0 to 100 in 1.8 milliseconds. |
| **5. Database** | MySQL 8.0 | The digital ledger that safely saves all transactions, fraud alerts, and settings. |

## Part 3: The Dataset (Why "PaySim" and Not Real Google Pay Data?)

> ⚠️ **Common Teacher Question: "Why didn't you collect real Google Pay or PhonePe data?"**
>
> **Say this with confidence:**
> *"Ma'am, real bank transactions contain private customer details like mobile numbers, bank balances, and UPI PINs. Under RBI privacy rules and the Indian Data Protection Law, no bank or app like PhonePe or Google Pay is legally allowed to share real customer transaction logs with students or the public."*

### What is PaySim?

- **What it is:** PaySim is an open academic dataset created by university researchers from real mobile money logs. It is used in fraud research worldwide.
- **Size:** It contains **63 lakh (6.36 million)** transactions.
- **Class Imbalance (The 99.87% Fact):**
  - Out of 63 lakh transactions, only **8,213 are fraud (0.13%)**.
  - **99.87% of transactions are completely honest!**
  - This is just like real life: on a normal day, out of 1,000 UPI payments, maybe only 1 is a fraud attempt.
- **Where Fraud Happens:** In PaySim, all fraud happens in only 2 payment types: `TRANSFER` (sending money to someone) and `CASH_OUT` (withdrawing money through an agent or ATM). Normal merchant payments (like paying at a tea stall) had 0 fraud. So our model focuses on these two risky types.

## Part 4: The 2 Smart Math Clues (Feature Engineering)

If you only give raw numbers (Amount, Balance) to an AI, it gets confused. We created **two simple math clues** that instantly reveal a thief.

> **Clue 1: Sender Balance Math ( `errorBalanceOrig` )**
>
> **The Basic Logic:**
> *Starting Balance - Money Sent = Ending Balance*
>
> - **In Normal Payments:** You have ₹10,000. You send ₹2,000 to a friend. Your new balance must be ₹8,000. Math: `New Balance (8,000) + Sent (2,000) - Old Balance (10,000) = 0`. **Error = 0 (Normal)**
> - **In Fraud (Account Drain):** A hacker breaks into an account with ₹1,00,000. They send ₹1,00,000 away! The victim's balance drops to **₹0** immediately. The math breaks completely, and our equation catches it instantly!

**Clue 2: Receiver Balance Math ( `errorBalanceDest` )**

**The Basic Logic:**

*Receiver Old Balance + Money Received = Receiver New Balance*

- **In Normal Payments:** The receiver's balance increases by the exact amount sent. The error is 0.
- **In Fraud (Mule Accounts):** Thieves send stolen money to newly made "mule" accounts and immediately withdraw it at an ATM so the police cannot freeze it. The receiver's balance shows an unnatural jump or stays at 0.

💡 **Teacher Trap: "How do you know the receiver's balance in real UPI?"**

**Answer:** *"Ma'am, FraudGuard is designed for the **Bank Central Switch / Core Banking System (like NPCI)**, which manages both sender and receiver banks. If the receiver is outside the network and their balance is hidden, our system sets it to 0 and treats it as an external unverified account."*

## Part 4B: The Parameters Behind the Simulator (Why These 8 Inputs?)

If the teacher points to the simulator and asks: "*What are all these parameters? Why did you pick these exact 8 inputs?*"

| Parameter Name | What It Holds | Why It Is Needed for Fraud Detection |
|---|---|---|
| `txn_type` | TRANSFER or CASH_OUT | PaySim data proved 100% of frauds happen in these 2 types. Normal payments (tea stall, grocery) have 0 fraud. |
| `amount` | ₹ Value of Transfer | Measures transaction volume. Big amounts raise attention, but amount alone is not enough to flag fraud. |
| `sender_id` & `receiver_id` | UPI IDs (e.g. user@upi) | Identifies the two parties to track previous history, account age, and known mule account lists. |
| `old_balance_org` & `new_balance_org` | Sender Balances Before & After | **The #1 Clue (Account Drainage):** Checks if the sender was wiped to ₹0. Fraudsters drain 100% of savings before the victim notices. |
| `old_balance_dest` & `new_balance_dest` | Receiver Balances Before & After | Tracks the destination. In an Account Drain attack, the receiver receives the ₹2,50,000 into their mule account (₹0 → ₹2,50,000). |

**The Hidden 10-Feature Vector:**

When the user clicks 'Run Risk Evaluation', the Django backend takes these 8 inputs, calculates the 2 math error clues ( `errorBalanceOrig` and `errorBalanceDest` ), and creates a **10-feature array** that is passed into `XGBoost.predict_proba()` in just 1.8 milliseconds!

## Part 5: What is XGBoost? (Explained Simply)

If the teacher asks: "*What algorithm did you use, and how does it work?*"

- We used **XGBoost (Extreme Gradient Boosting)**.
- **Step 1 (A Single Decision Tree):** Think of a simple 20-questions game:
  "*Is the amount greater than ₹50,000?*" $\rightarrow$ "*Did sender balance become zero?*" $\rightarrow$ "*Yes? Then it's fraud!*"
  A single tree can make mistakes.
- **Step 2 (Boosting = Teamwork):** Instead of 1 tree, XGBoost creates a **team of 100 trees, one after another**:
  - Tree #1 makes its guesses. It gets some transactions wrong.
  - Tree #2 is built specifically to **learn and fix the mistakes made by Tree #1**.
  - Tree #3 fixes the mistakes of Tree #2.
- In the end, all 100 trees vote together, giving an ultra-accurate **Risk Score from 0 to 100**.

> ⚠️ **Teacher Trap: "Why didn't you use Deep Learning or Neural Networks?"**
>
> **Give these 3 simple reasons:**
>
> 1. **Trees are better for tables of numbers:** In machine learning research, decision tree models consistently beat deep neural networks when dealing with tabular numbers (amounts, balances).
> 2. **Speed (Latency):** In UPI, payments must complete in under 2 seconds. XGBoost takes only **1.8 milliseconds** on a normal CPU. Neural networks take 100 milliseconds and need expensive graphics cards (GPUs).
> 3. **Explainability:** Banks cannot use black boxes. XGBoost works directly with SHAP to explain reasons.

## Part 6: What is SHAP? (Why does the website show Red and Green reasons?)

If the teacher points to the Simulator and asks: "*What are these reasons below the gauge?*"

> **Why Banks Hate "Black Box" AI:**
>
> Imagine your bank blocks your card while you are buying groceries. You call customer service: "*Why did you block my card?*"
> If the bank says: "*The computer AI decided it, but we don't know why*", you would be furious! By banking law (RBI), banks **must explain why** a transaction was blocked.

- **SHAP** stands for **SH**apley **A**dditive ex**P**lanations.
- It calculates exactly how much points each clue contributed:
  - `Red Badge (Raises Risk)`: e.g., *Sender Balance Drained to 0 (+0.45)*, *Big Transfer Amount (+0.25)*.
  - `Green Badge (Lowers Risk)`: e.g., *Normal receiver balance growth (-0.15)*.
- This turns a "black box" into an **open, explainable system**!

## Part 7: Why "Accuracy" is a TRAP in Fraud Detection!

⚠️ **The #1 Question Teachers Love to Ask: "What is your accuracy? Is 99% accuracy good?"**

**Tell this exact story to your teacher:**

> *"Imagine 1,000 patients visit a doctor for a very rare disease that only 1 person has.*
> *The doctor is lazy and tells EVERY single patient: 'You are healthy, go home!' without doing any test.*
> *Did the doctor guess 999 out of 1,000 patients right?* ***YES! His accuracy is 99.9%!***
> *But did he save the sick person?* ***NO! The patient died. His fraud-catching score is ZERO!***
>
> *In fraud detection, 99.87% of payments are honest. If our program just approves every payment, it gets* ***99.87% accuracy****, but catches ZERO thieves! That is why Accuracy is a trap."*

### The 4 Real Scores that Actually Matter:

| Score Name | Our Result | Simple Meaning | Real-Life Analogy |
|---|---|---|---|
| **Recall** (Catch Rate) | 99.57% | Out of 100 real thieves, how many did we catch? (We caught 99.5%). | **Airport Metal Detector:** It must catch 100% of weapons. We cannot afford to miss even one knife! |
| **Precision** (Trust Rate) | 98.79% | When our alarm rings, how often is it a real thief vs an innocent customer? | **Police Arrests:** If police arrest 100 people, 98.8 are real criminals and only 1.2 are innocent false alarms. |
| **F1-Score** | 99.18% (0.992) | A fair average that balances Recall and Precision together. | A student scoring 99 in Science and 98 in Math has a balanced top rank. |
| **PR-AUC** | 99.84% (0.998) | The overall score across both strict and lenient modes. | The final gold-standard grade in banking AI research. |

## Part 8: What is the Threshold Knob in Settings?

If the teacher asks: "*What is that slider on your settings page?*"

- The AI calculates a risk score from 0 to 100%. The slider is our **alert trigger line**:
- **If you slide it DOWN (e.g. 30%):**
  The system becomes ultra-strict. It catches every single thief (High Recall), but it might stop a few honest customers by mistake (Lower Precision).
  *Use case: During high-risk Diwali sales or cyber-attack alerts.*
- **If you slide it UP (e.g. 80%):**
  The system only rings when it is 100% sure. Honest customers are never disturbed, but a small sneaky thief might slip through.
- **Our Default is 50%:** The sweet spot with balanced F1-score.

## Part 9: Top 10 Random Teacher Questions & Simple Answers

**Q1: Where is your model saved, and how does Django use it?**
**Answer:** It is saved at `ml/artifacts/model.joblib`. When Django starts, a small Python script (`transactions/model_loader.py`) loads it into computer memory once, so it predicts instantly without reloading.

**Q2: How fast is your system? What is the latency?**
**Answer:** The machine learning model takes **1.8 milliseconds**. The full website request (including database checks) takes only **15 to 25 milliseconds**, which is well within UPI's 2-second limit.

**Q3: What database did you use, and what tables exist?**
**Answer:** We used **MySQL 8.0** with 4 tables: `Transaction` (all payments), `FraudAlert` (flagged fraud cases), `ModelMetric` (test scores), and `RiskSetting` (the threshold slider value).

**Q4: How do you prevent hackers from attacking your website?**
**Answer:** Django ORM prevents SQL injection automatically by parameterizing all queries. For web form security, all requests require Django's CSRF token.

**Q5: What do the 4 Preset Buttons on the Simulator do?**
**Answer:** They let us show real-life tests in one click:
1. *Account Drain:* Simulates transferring victim's entire balance to a fraudster account (shows 100% Red Risk).
2. *Drain + Cash-Out:* Simulates withdrawing all funds as physical cash leaving 0 digital balance (shows 100% Red Risk).
3. *Normal UPI Transfer:* Routine ₹5,000 transfer (shows 0% Green Safe).
4. *Small Cash-Out:* Normal ₹2,000 ATM withdrawal (shows 0% Green Safe).

**Q6: What happens when an analyst clicks "Mark as Reviewed" on an alert?**
**Answer:** It provides a Human-in-the-Loop review. A human officer confirms the fraud and clicks "Reviewed". This sends a quick `PATCH` request to the Django API and updates the alert status in MySQL.

**Q7: Why XGBoost instead of Random Forest?**
**Answer:** Random Forest builds trees independently. XGBoost builds trees step-by-step, where each new tree fixes the errors of the previous ones. XGBoost is 4x faster and scored higher on our dataset.

**Q8: How did you test the model without cheating? (Data Leakage)**
**Answer:** We split the data into 80% Training and 20% Testing before calculating any averages or equations. The test set was kept completely untouched until final evaluation.

**Q9: Can this run in Docker?**
**Answer:** Yes, Ma'am. We created a production `Dockerfile` and `docker-compose.yml` so the whole system, including Django and MySQL, can run in a clean container.

**Q10: What is your project's conclusion in one line?**
**Answer:** FraudGuard proves that by combining smart balance equations with XGBoost, banks can stop UPI fraud in under 2 milliseconds without acting like an unexplainable black box.

## Part 10: 5-Minute Team Presentation Script

Here is how you can divide the presentation among the 4 team members so everyone speaks smoothly:

**Member 1 (Wasil - Team Lead): Introduction (~1 Minute)**

*"Good morning Ma'am. Our project is **FraudGuard** — a Real-Time UPI Fraud Detection System. India processes billions of UPI transactions every month, and hackers are increasingly using account takeover attacks to drain victims' bank accounts. Old bank rules like 'flag transactions above ₹50,000' fail because hackers simply send smaller amounts. We built an intelligent system that evaluates transactions in under 2 milliseconds using Machine Learning and Explainable AI. I will now hand over to Tayyab to explain the dataset and our AI model."*

**Member 2 (Tayyab - Data & Machine Learning): (~1.5 Minutes)**

*"Thank you, Wasil. Because real bank logs are confidential under RBI privacy laws, we used the internationally recognized PaySim dataset containing 63 lakh transactions. In fraud, 99.87% of transactions are honest, and only 0.13% are fraud. Rather than feeding raw amounts, we engineered two smart balance equations: Sender Balance Error and Receiver Balance Error. When a hacker empties an account, this ledger math deviates to zero. We trained an XGBoost model that achieved 99.57% Recall and 98.79% Precision. Haseeb will now demonstrate our live website."*

**Member 3 (Haseeb - Live System Demo): (~1.5 Minutes)**

*"Thank you, Tayyab. Here is our live FraudGuard console. On our Transaction Simulator, I will click 'Account Drain Attack'. You can see ₹2,50,000 is sent, wiping the sender balance to zero. When I click Run Evaluation, in less than 2 milliseconds our gauge jumps to 98% Risk, flagging the fraud! Below, our SHAP Explainable AI explains in plain English that 'Sender Balance Discrepancy' was the main reason. Now, if I click 'Normal UPI Transfer', the gauge stays at 0% Safe. Flagged attacks automatically appear in our Fraud Alerts queue for bank officers to review. Sufiyan will now explain our metrics."*

**Member 4 (Sufiyan - Metrics & Conclusion): (~1 Minute)**

*"Thank you, Haseeb. On our Analytics page, we track daily trends. In fraud detection, standard Accuracy is a trap because a useless model that approves everything can still get 99.87% accuracy while missing all fraud. We therefore benchmarked on Recall (99.57%) and F1-Score (99.18%). On our Settings page, bank officers can adjust the sensitivity threshold slider to balance fraud catching against false alarms. In conclusion, FraudGuard provides a fast, sub-2ms, transparent shield for digital payments. We are now happy to take your questions."*